Creating a static Chinese Wall in SELinux

The Chinese Wall policy states that a subject s_1 can operate on objects o_1 and o_2, where o_1 and o_2 are in the same CoI group.

A simple daemon process to open /var/log/messages

```
#include <unistd.h>
#include <stdio.h>

FILE *f;

int main(void){
  while(1){
    f = fopen("/var/log/messages","w");
    sleep(5);
    fclose(f);
  }
}
```

The result of ls –Z on /var/log

```
system_u:object_r:var_log_t:s0 log
```

The result of ls –Z on mydaemon

```
cal@localhost:/home/testgrounds/daemon$ ls -Z
unconfined_u:object_r:user_home_dir_t:s0 mydaemon  unconfined_u:object_r:user_home_dir_t:s0 mydaemon.c  unconfined_u:object_r:user_home_dir_t:s0 mydaemon.service
```

This is just the labels on the executable, not the actual service

```
cal@localhost:/home/testgrounds/daemon$ sudo systemctl start mydaemon
[sudo] password for cal:
cal@localhost:/home/testgrounds/daemon$ sudo journalctl | grep mydaemon
Jan 13 16:33:34 localhost.localdomain sudo[3968]:      cal : TTY=pts/0 ; PWD=/home/testgrounds/daemon ; USER=root ; COMMAND=/bin/systemctl start mydaemon
Jan 13 16:33:34 localhost.localdomain systemd[1]: Started mydaemon.service - Simple testing daemon.
cal@localhost:/home/testgrounds/daemon$
```

As it is running as unconfined_u, it is not restricted by SELinux.

```
cal@localhost:/home/testgrounds/daemon$ ps -efZ | grep mydaemon
system_u:system_r:mydaemon_t:s0 root        3989       1  0 16:33 ?        00:00:00 /usr/local/bin/mydaemon
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 cal 4137 3484  0 16:46 pts/0 00:00:00 grep --color=auto mydaemon
cal@localhost:/home/testgrounds/daemon$
```

```Sepolicy generate —init /usr/local/bin/mydaemon``` autogenerates a policy to run the daemon confined by SELinux
```

Policy generated by SELinux

```
policy_module(mydaemon, 1.0.0)

########################################
#
# Declarations
#

type mydaemon_t;
type mydaemon_exec_t;
init_daemon_domain(mydaemon_t, mydaemon_exec_t)

permissive mydaemon_t;

########################################
#
# mydaemon local policy
#
allow mydaemon_t self:fifo_file rw_fifo_file_perms;
allow mydaemon_t self:unix_stream_socket create_stream_socket_perms;

domain_use_interactive_fds(mydaemon_t)

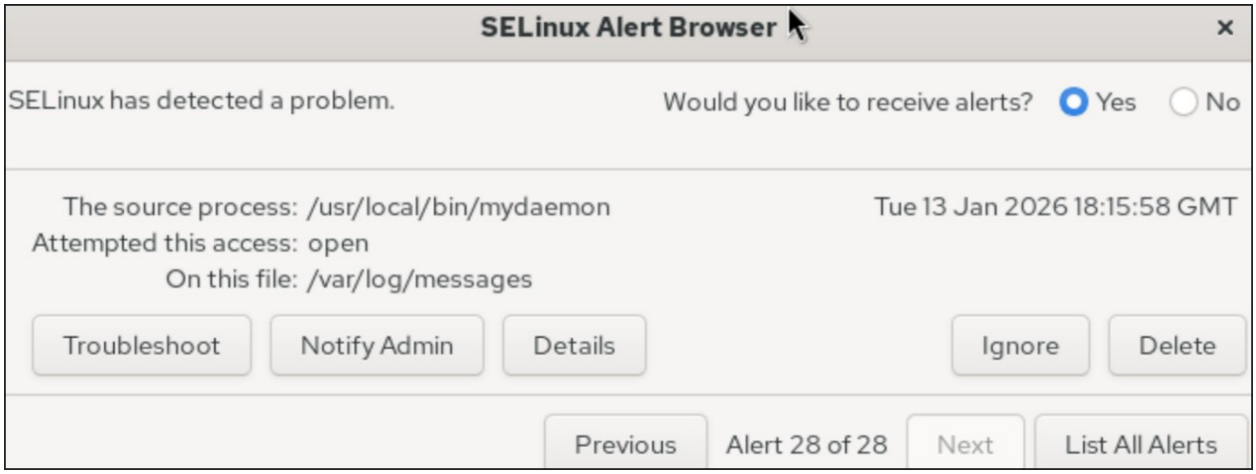
files_read_etc_files(mydaemon_t)

miscfiles_read_localization(mydaemon_t)
```

Once reloaded, the daemon is now running under the mydaemon_t type declared in the file

```
cal@localhost:/home/testgrounds/daemon$ systemctl daemon-reload
cal@localhost:/home/testgrounds/daemon$ ps -efZ | grep mydaemon
system_u:system_r:mydaemon_t:s0 root       62739       1  0 17:28 ?        00:00:00 /usr/local/bin/mydaemon
unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 cal 62983 3484  0 17:28 pts/0 00:00:00 grep --color=auto mydaemon
cal@localhost:/home/testgrounds/daemon$
```

SELinux denies access, as there is no rule to allow mydaemon_t to read / write in var/messages



Adding these lines to the policy file allow it to work under SELinux

```
type mydaemon_t;
type mydaemon_exec_t;
type var_log_t;
init_daemon_domain(mydaemon_t, mydaemon_exec_t)

permissive mydaemon_t;

########################################
#
# mydaemon local policy
#
allow mydaemon_t self:fifo_file rw_fifo_file_perms;
allow mydaemon_t self:unix_stream_socket create_stream_socket_perms;
allow mydaemon_t var_log_t:file { open write getattr };

domain_use_interactive_fds(mydaemon_t)

files_read_etc_files(mydaemon_t)

miscfiles_read_localization(mydaemon_t)
```

Now these fundamentals can allow us to create a static Chinese Wall

A static wall can be displayed with 2 different subjects, s_1 and s_2 being able to view different items in the same CoI class but not another. A chinese wall is better made with the subject being allowed to view all and then be restricted from the object they are disallowed from rather than being allowed to view the other object. SELinux policy modules have a neverallow feature that can be used for this

```
/home/testgrounds/static ❯ sudo systemctl status bar_daemon
● bar_daemon.service - Bar's daemon
     Loaded: loaded (/usr/lib/systemd/system/bar_daemon.service; disabled; preset: disabled)
     Active: active (running) since Tue 2026-01-13 19:44:30 GMT; 4s ago
 Invocation: 3d2424dbed264648a5728eb359d1decb
   Main PID: 67243 (bar_daemon)
      Tasks: 1 (limit: 35903)
     Memory: 180K (peak: 1.2M)
        CPU: 2ms
     CGroup: /system.slice/bar_daemon.service
             └─67243 /usr/local/bin/bar_daemon

Jan 13 19:44:30 localhost.localdomain systemd[1]: Started bar_daemon.service - Bar's daemon.
```

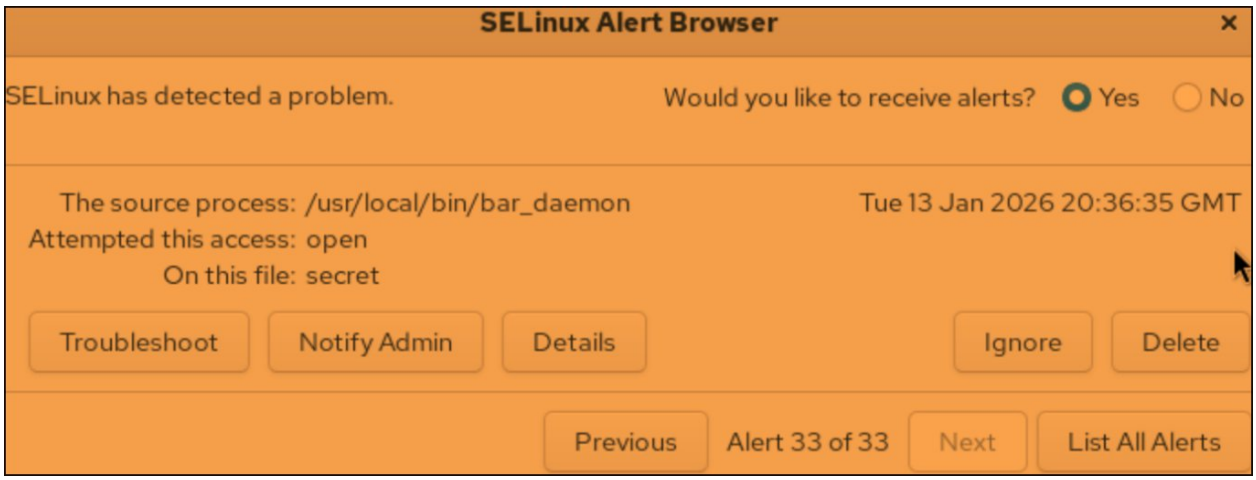
Created a bar daemon to try read foo_secret, works currently as its unconfined by SELinux

```
/home/testgrounds/static 59s ❯ make -f /usr/share/selinux/devel/Makefile bar_daemon.pp
Compiling targeted bar_daemon module
Creating targeted bar_daemon.pp policy package
rm tmp/bar_daemon.mod.fc tmp/bar_daemon.mod

/home/testgrounds/static ❯ sudo semodule -i bar_daemon.pp

/home/testgrounds/static ❯
```

Policy created for bar_daemon to restrict access to secret_t (defined in another policy module)



Bar Daemon is denied from accessing secret while foo still can. This is the preliminary functionality to implement a Chinese wall.